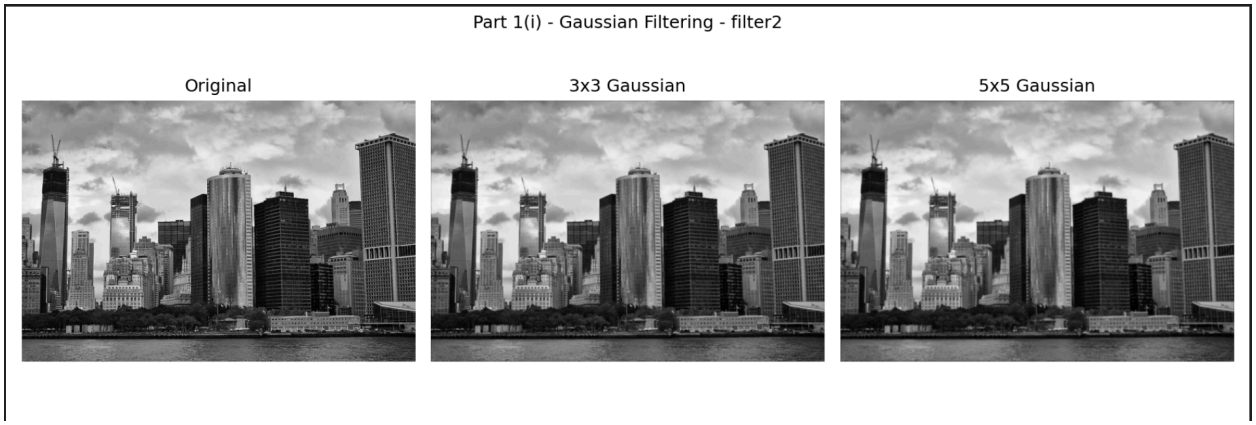
CS 410 Computer Vision and Deep Learning
Programming Assignment #1 Write-Up
Veniamin Velikoretskikh
veniamin@pdx.edu

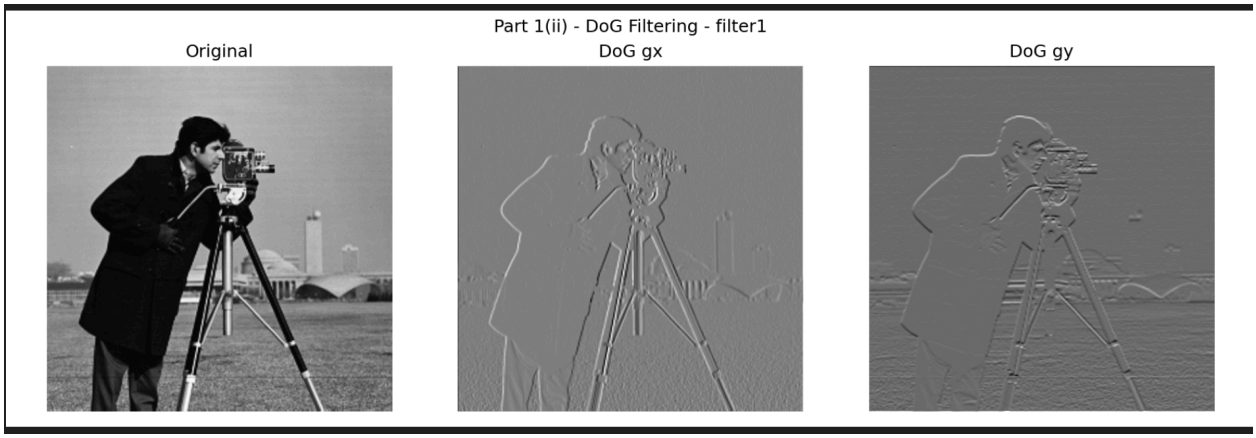
Part 1

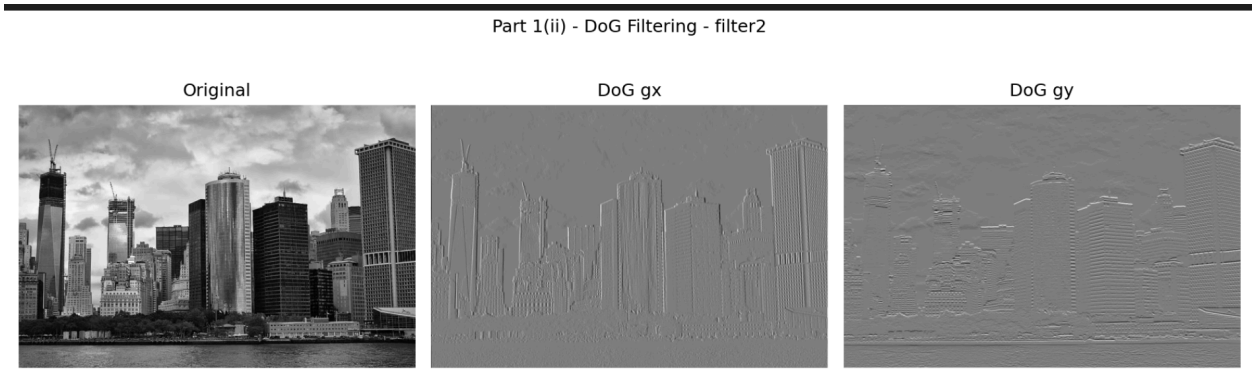
For this part, I made image filtering using 2d convolution function. I used 3 filter types to two images. Gaussian smoothing 3 by 3, 5 by 5, DoG, and Sobel filters.



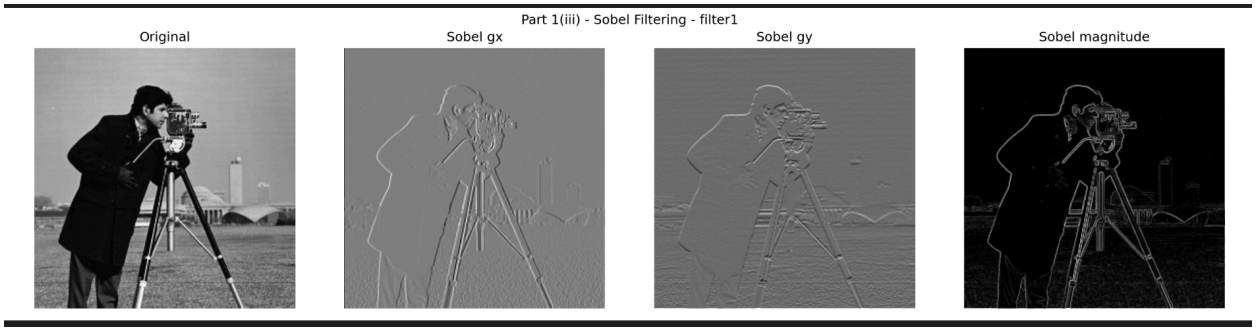


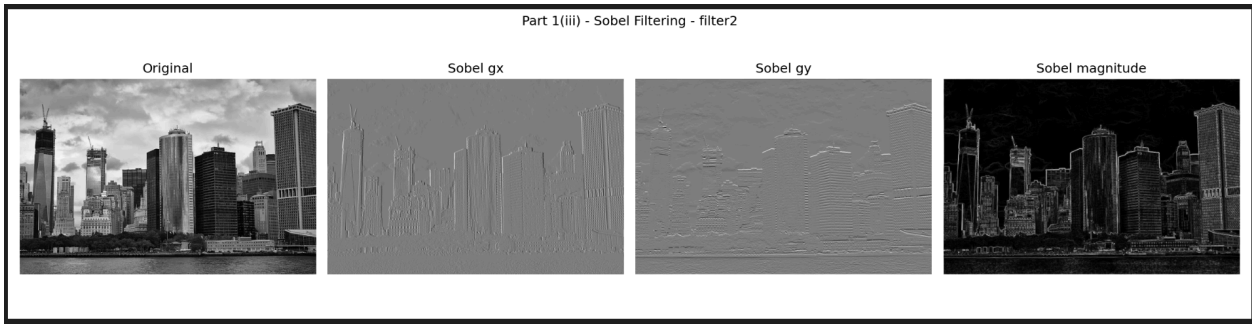
Both 3 by 3 and 5 by 5 blur the high-frequency detail but preserve the overall structure. The 5 by 5 makes more blur, and you can see that it smooths the image more. I can see that the sky is smoother with less of the sharp detail.





The gx filter, horizontal gradient, traces out more of the vertical edges. I can see that wil highlighting the horizontal lines, like the tripod for the camere of the building edges. The gy fileter the vertical gradient will be the opposite and highlight the horizontal features. Like the grass and the window rows anf rooftops for the skyline picture.





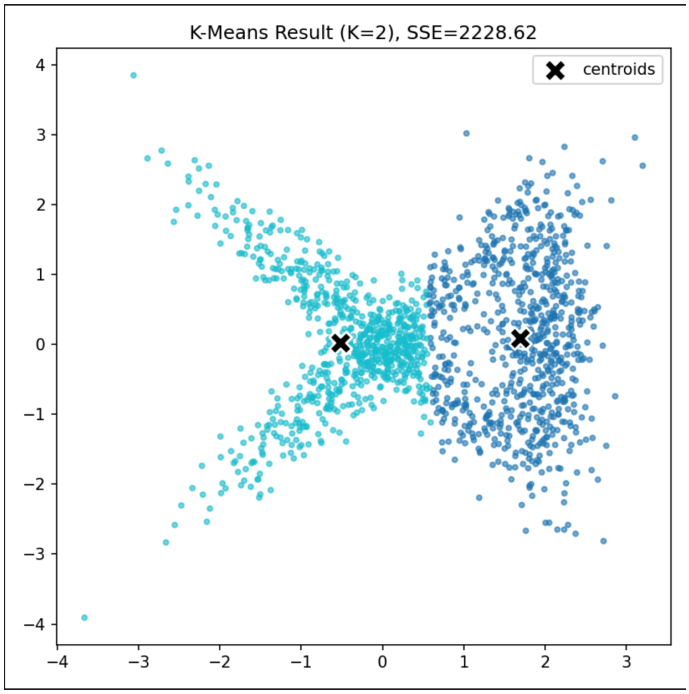
The Sobel gx/gy filter does the same thing as the DoG filter. But the Sobel filter is sharper and less noisy with its lines. It combines the horizontal and vertical features into 1 picture, highlighting the most important features and reducing the sensitivity to small pixel noise.

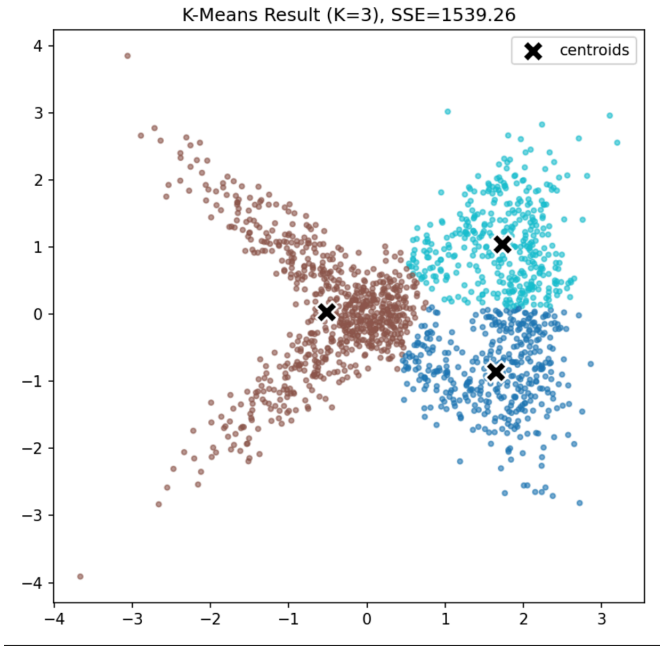
This is Part 2: K-Means Clustering

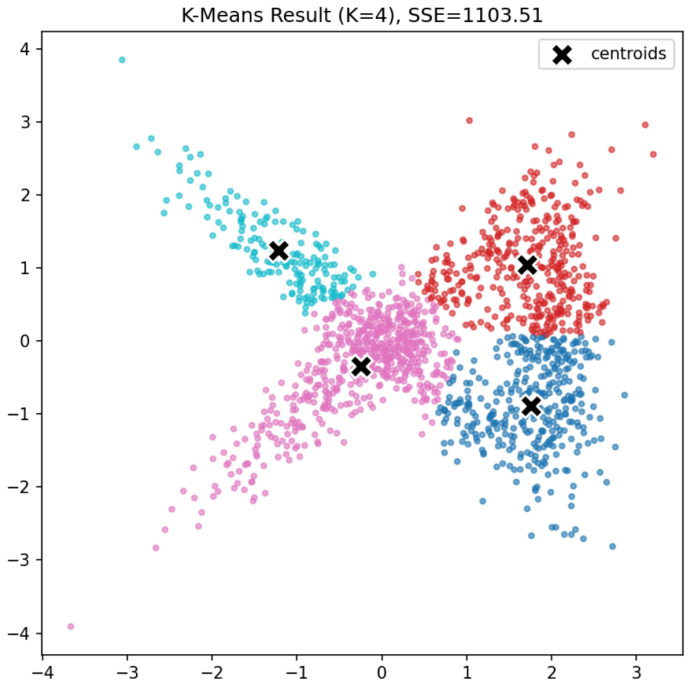
K-means clustering picks a random data point for the initial centroid. Points are then assigned to the nearest centroid. The centroids are updated to the mean of their assigned points. This will repeat until there's convergence.

Also, for part 2 (i), 10 random restarts were run per K value. It will keep the restart with the lowest sum of squares error.

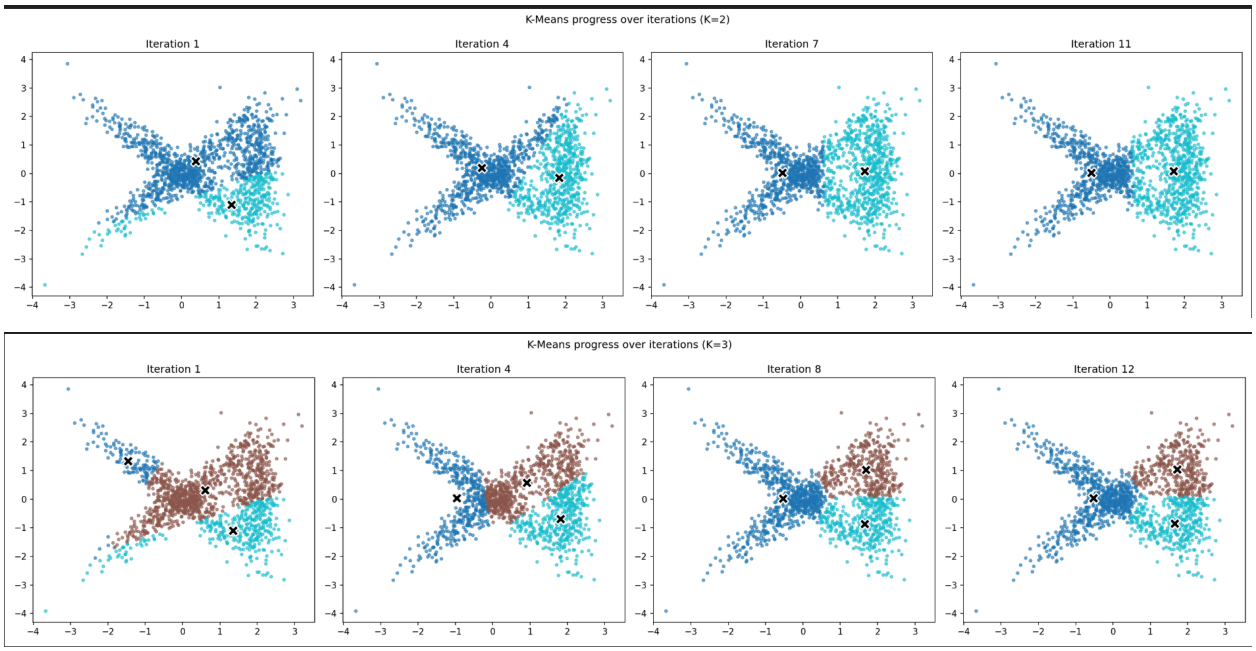
K	Best SSE of r = 10 restarts
2	2228.62
3	1539.26
4	1103.51

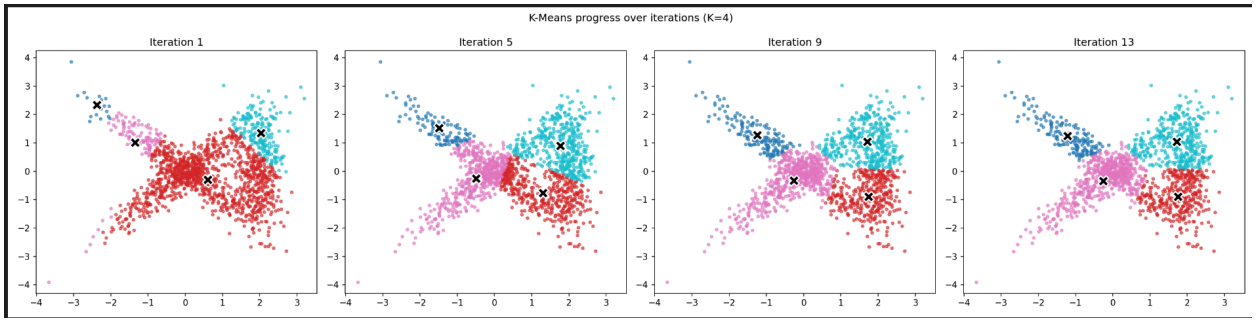

K-Means Result (K=2), SSE=2228.62
centroids
4
3
2
1
0
−1
−2
−3
−4
−4
−3
−2
−1
0
1
2
3


K-Means Result (K=3), SSE=1539.26
centroids
4
3
2
1
0
−1
−2
−3
−4
−4
−3
−2
−1
0
1
2
3



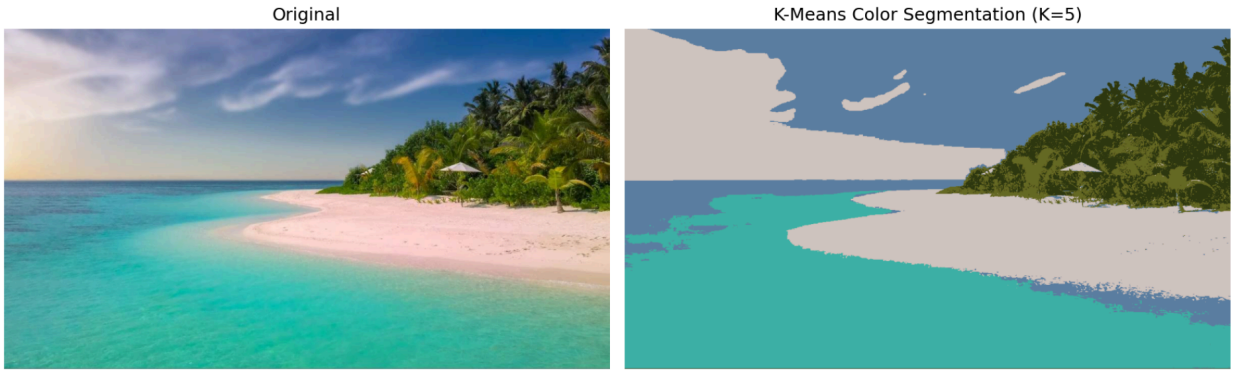
For 2 clusters, K-means divides the data into 2 halves, giving a very rough approximation that mixes points from different source distributions in the same cluster and has the highest SSE.
For 3 clusters, the data is split a bit better, and the SSE drops significantly to 1539.26, compared to 2228.62 in 2 clusters.
For 4 clusters, the SSE drops even more to 1103.51. It groups the data a lot more, but it doesn't always mean it's better with more clusters.





For all of the 3 different k values, the centroids start at randomly chosen locations. Then it migrates toward the center of the points assigned to the section. This keeps on repeating until there's no more significant movement of the centroid.

2 (ii) k-means color segmentation



For the RGB, k = 5, the image still holds most of its structure. But it’s only split into a couple of different colors, smoothing out a lot of the detail.



With double the amount of clusters, there is noticeably more detail. More shades of color and more details are visible, making it a lot closer to the original image.

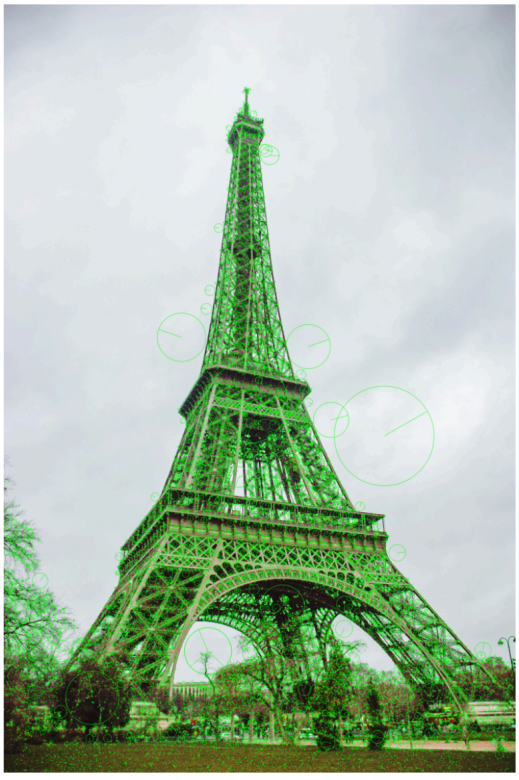
Part 3 SIFT

The SIFT keypoints and 128-dimensional descriptors were detected using the OpenCV library. Then computed the L2 distance between the SIFT descriptor vectors between 2 images. Then I kept only the 10% of the matches that were closer together.

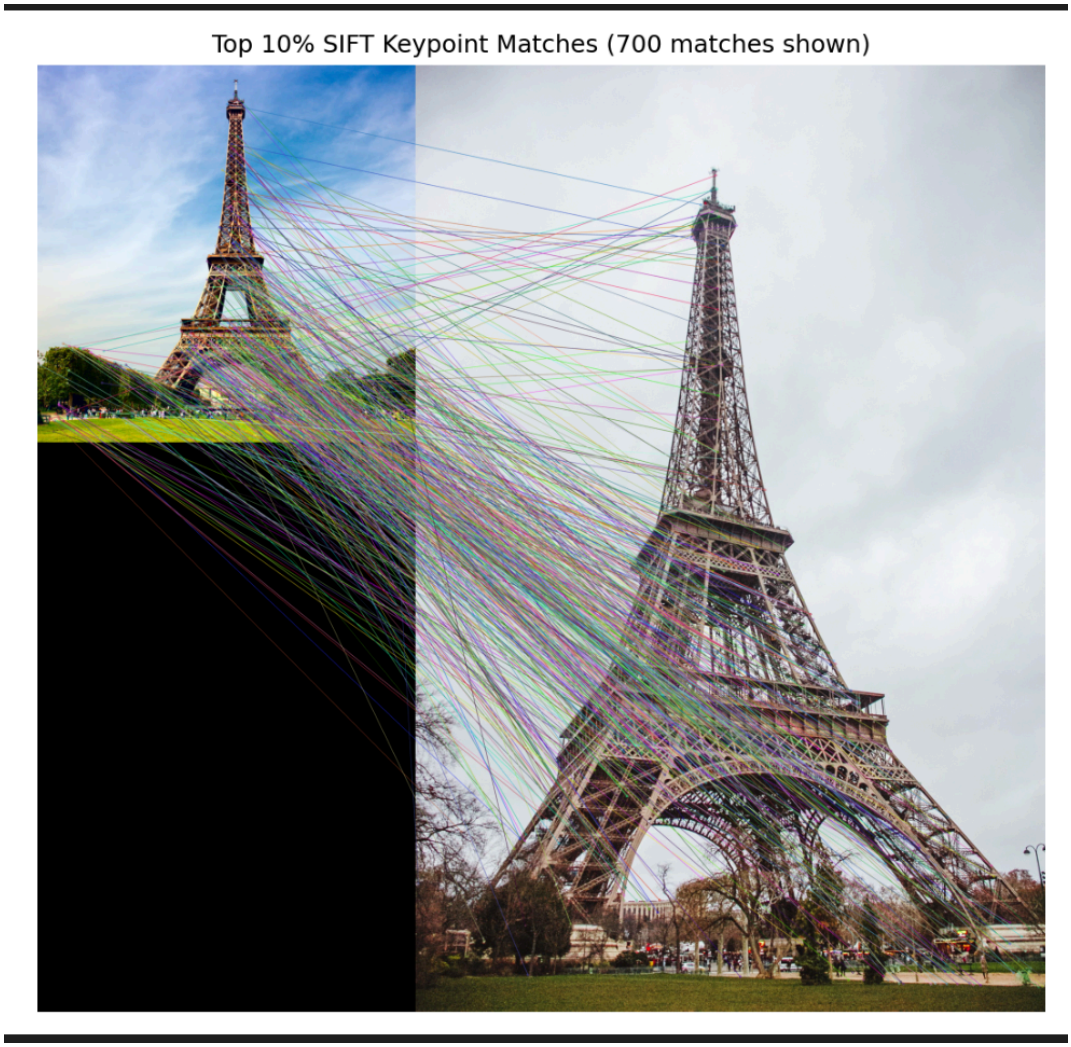
SIFT Keypoints - Image 1 (7005 keypoints)



SIFT Keypoints - Image 2 (41380 keypoints)



The green dots are all of the keypoints detected. The smaller the dot, the finer the keypoint; the larger, the coarser the keypoint. The second picture has significantly more keypoints compared to the first image.



This picture shows the kept 10% of the keypoints, which comes out to 700. It only kept the closest keypoints by using the L2 distance. The lines connect similar keypoints from the 2 images. Even though the images are taken from different angles. I can still see that some of the keypoints still don't match up properly. Like the top of the tower matches the bottom for some reason.